

Queue

A queue is a linear data structure that stores elements in a particular order and follows the First In, First Out (FIFO) principle. This means that the element that enters the queue first is the first element to leave. Unlike a stack, where insertion and removal happen at the same end, a queue normally adds elements at the rear (end) and removes elements from the front (start).

A simple real-world example is a line of people waiting at a checkout counter. The person who arrives first is served first, while new people join at the back of the line. This same idea is used in many computer systems, such as printer job scheduling, task processing, and waiting requests.

For example, if we enqueue 10, 20, and 30, the queue becomes:

Front $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\leftarrow$ Rear.

If we dequeue an element, 10 is removed first. The next dequeue removes 20, followed by 30.

Therefore, the removal order is the same as the insertion order: 10 $\rightarrow$ 20 $\rightarrow$ 30.

A printer queue can contain several print jobs. The first job sent to the printer is normally processed before the jobs that arrive later.

Queue Operations

1. enqueue()

The enqueue() operation adds a new element to the rear/end of the queue. The rear position is advanced and the new value is stored there. If the queue is full in a fixed-size implementation, the new element cannot be inserted and queue overflow occurs.

Before: Front $\rightarrow$ 10 $\rightarrow$ 20 $\leftarrow$ Rear

enqueue(30)

After: Front $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\leftarrow$ Rear

Time Complexity: $O(1)$ in the basic array/circular-array implementation.

2. dequeue()

The dequeue() operation removes the element at the front/start of the queue. After removal, the front position moves to the next available element. Because queues follow FIFO, the oldest element is always removed first.

Before: Front $\rightarrow$ 10 $\rightarrow$ 20 $\rightarrow$ 30 $\leftarrow$ Rear

dequeue()

After: Front $\rightarrow$ 20 $\rightarrow$ 30 $\leftarrow$ Rear

Time Complexity: $O(1)$

3. isEmpty()

isEmpty() checks whether the queue currently contains any elements. It returns true when the queue has no elements and false when at least one element is present. This check is useful before performing dequeue().

Empty queue $\rightarrow$ isEmpty() = true

Queue with elements $\rightarrow$ isEmpty() = false

Time Complexity: $O(1)$

4. isFull()

isFull() checks whether a fixed-size queue has reached its maximum capacity. It is especially important for a static array implementation because there is a limited number of array positions.

Capacity = 5

Current elements = 5

isFull() = true

Time Complexity: $O(1)$

5. displayAll()

displayAll() is a helper operation used to display all elements currently stored in the queue.

The elements are normally displayed from the front to the rear so that their processing order is clear. Displaying the elements does not remove them from the queue.

Front $\rightarrow$ 10 20 30 $\leftarrow$ Rear

Output: 10 20 30

Time Complexity: $O(n)$, because every stored element may need to be visited.

Front/Start and Rear/End

A queue normally uses two positions to keep track of its ends. The front, sometimes called start, identifies the element that will be removed next. The rear, sometimes called end, identifies the position where the next new element will be inserted.

Index: 0 1 2 3 4

Array: [10] [20] [30] [] []

↑

Front

↑

Rear

After enqueue(40), the rear moves to the next position and 40 is inserted. After dequeue(), the front moves forward because the previous front element has been removed.

enqueue(40): [10] [20] [30] [40] []
 ↑ ↑
 Front Rear

dequeue(): [10 removed] [20] [30] [40]
 ↑ ↑
 Front Rear

The exact meaning of an empty queue depends on the implementation. In a common array implementation, front and rear are initialized to values that indicate no active elements. In a circular queue, the indexes are updated using modulo arithmetic.

Queue Overflow and Underflow

Queue Overflow

Queue overflow occurs when an enqueue() operation is attempted on a queue that is already full. With a fixed-size array, there is no available position for the new element.

Capacity = 3

Front → [10] [20] [30] ← Rear

enqueue(40) → QUEUE OVERFLOW

Overflow is mainly a concern with fixed-size array implementations. A dynamically allocated queue does not have the same fixed capacity, although memory can still become exhausted.

Queue Underflow

Queue underflow occurs when dequeue() is attempted on an empty queue. There is no element available to remove. A program should check isEmpty() before dequeue() to avoid this condition.

Queue: [empty]

dequeue() → QUEUE UNDERFLOW

The important difference is simple: overflow happens when trying to add to a full queue, while underflow happens when trying to remove from an empty queue.

Linear Array Queue and Unused Locations Problem

A simple linear queue can be implemented using a fixed-size array with front and rear indexes. The problem is that, after dequeue operations, the empty positions at the beginning of the array may no longer be reused by the rear.

Initial:

```
[10] [20] [30] [40] [50]
  ↑           ↑
Front       Rear
```

After two dequeue operations:

```
[ ] [ ] [30] [40] [50]
      ↑   ↑
    Front Rear
```

At this point, the first two array positions are empty. However, if the rear has already reached the last array index, a simple linear implementation may report the queue as full even though unused positions exist at the beginning. This is sometimes called false or wasted overflow.

One way to solve this is to shift all remaining elements toward the beginning, but shifting takes $O(n)$ time and adds unnecessary work. A better approach for a fixed-size array is a circular queue.

Circular Queue and Wrap-Around

A circular queue treats the array as if its last position is connected back to its first position. When the rear reaches the last array index, the next position can be index 0 if that position is available. This allows the queue to reuse locations that were freed by dequeue operations.

Array indexes:

```
0  1  2  3  4
[] [] [] [] []
  ↑       |
  └──────┘
```

The index wraps around using modulo arithmetic. If the array capacity is 5, the next index after 4 is calculated as $(4 + 1) \% 5 = 0$. In the same way, an index can continue from 0 to 1, 2, and so on.

next position = (current position + 1) % capacity

$4 \rightarrow (4 + 1) \% 5 \rightarrow 0$

$0 \rightarrow (0 + 1) \% 5 \rightarrow 1$

For example, after elements at the front are removed, a new enqueue can place an element in one of those freed positions instead of leaving the space unused.

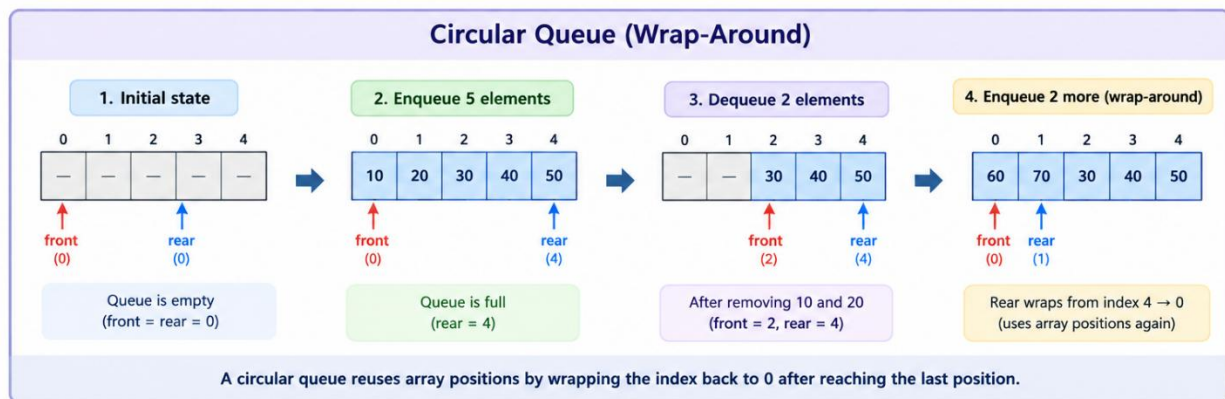
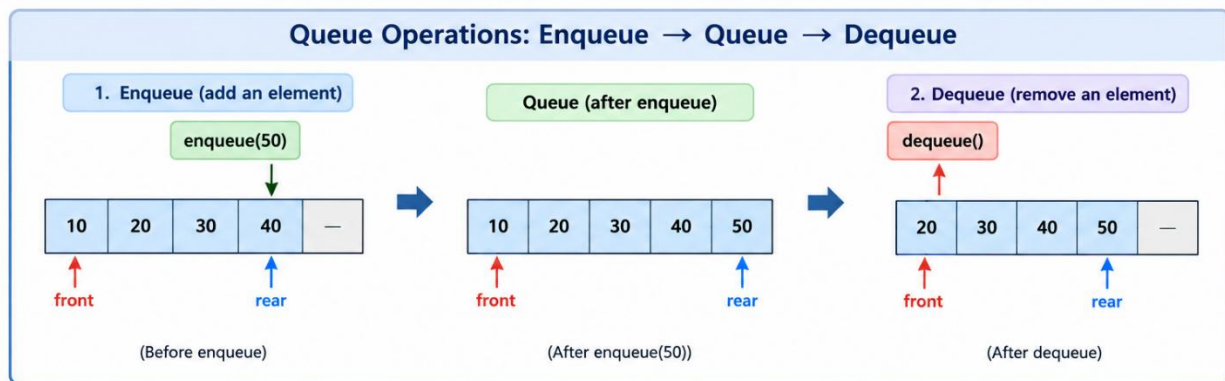
Before wrap-around:

[50] [] [30] [40] [50?]

Conceptual circular movement:

... index 4 $\rightarrow$ index 0 $\rightarrow$ index 1 ...

A circular queue therefore makes better use of a fixed array because positions can be reused as elements leave the queue. Enqueue and dequeue can remain $O(1)$ when the indexes are maintained correctly.



Array-Based Circular Queue

```
1  #include <iostream>
2  using namespace std;
3
4  class Queue {
5  private:
6      const static int SIZE = 5;
7
8      int array[SIZE];
9      int start;
10     int end;
11     int count;
12
13 public:
14     Queue() {
15         start = 0;
16         end = -1;
17         count = 0;
18     }
19
20     bool isEmpty() {
21         return count == 0;
22     }
23
24     bool isFull() {
25         return count == SIZE;
26     }
27
28     void enqueue(int value) {
29         if (isFull()) {
30             cout << "Queue Overflow" << endl;
31             return;
32         }
33
34         end = (end + 1) % SIZE;
35         array[end] = value;
36         count++;
37
38         cout << value << " enqueued." << endl;
39     }
40
41     void dequeue() {
42         if (isEmpty()) {
43             cout << "Queue Underflow" << endl;
44             return;
45         }
46
47         cout << array[start] << " dequeued." << endl;
48
49         start = (start + 1) % SIZE;
50         count--;
51     }
52
53     void displayAll() {
54         if (isEmpty()) {
55             cout << "Queue is Empty" << endl;
56             return;
57         }
58
59         cout << "Queue: ";
60
61         for (int i = 0; i < count; i++) {
62             int index = (start + i) % SIZE;
63             cout << array[index] << " ";
64         }
65
66         cout << endl;
67     }
68 };
69
70 int main() {
71     Queue q;
72
73     // Fill the queue
74     q.enqueue(10);
75     q.enqueue(20);
76     q.enqueue(30);
77     q.enqueue(40);
78
79     q.displayAll();
80
81     // Remove two elements
82     q.dequeue();
83     q.dequeue();
84
85     q.displayAll();
86
87     // These operations cause end to reach index 4
88     q.enqueue(50);
89
90     // This operation causes end to wrap from index 4 to index 0
91     q.enqueue(60);
92
93     q.displayAll();
94
95     return 0;
96 }
```